Practical 13

Part A:

Aim:

1. Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.

Code:

GenericContainers.java

```
class Box<T> {
    private T value;

    public Box(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }

    public void set(T value) {
        this.value = value;
    }
}

class Pair<A, B> {
    private final A first;
    private final B second;

    public Pair(A first, B second) {
```

```
        this.first = first;
        this.second = second;
    }

    public A getFirst() {
        return first;
    }

    public B getSecond() {
        return second;
    }
}

public class GenericContainers {

    // Generic method that returns the first element of any
array
    public static <T> T first(T[] array) {
        if (array == null || array.length == 0) {
            throw new IllegalArgumentException(
                    "Array must not be null or empty");
        }

        return array[0];
    }

    public static void main(String[] args) {
        // Box with different types
        Box<Integer> intBox = new Box<>(100);
        Box<String> stringBox = new Box<>("Hello Java");

        System.out.println("Integer Box: " + intBox.get());
        System.out.println("String Box: " + stringBox.get());

        intBox.set(200);
```

```
        System.out.println("Updated Integer Box: " +
intBox.get());

        // Pair with two different types
        Pair<String, Integer> student =
                new Pair<>("Riya", 101);

        System.out.println("Student Name: " +
student.getFirst());
        System.out.println("Student ID: " +
student.getSecond());

        // Generic method with different array types
        Integer[] numbers = {10, 20, 30};
        String[] names = {"Amit", "Riya", "Sara"};
        Double[] prices = {12.5, 25.0, 37.5};

        System.out.println("First number: " + first(numbers));
        System.out.println("First name: " + first(names));
        System.out.println("First price: " + first(prices));
    }
}
```

Output:



2. Bounded stats: a generic method <N extends Number> double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

Code:

BoundedStat.java

```
public class BoundedStats {

    public static <N extends Number> double average(N[] values)
{
        if (values == null || values.length == 0) {
            throw new IllegalArgumentException(
                    "Array must not be null or empty");
        }

        double sum = 0.0;

        for (N value : values) {
            if (value == null) {
                throw new IllegalArgumentException(
                        "Array must not contain null values");
            }

            sum += value.doubleValue();
        }

        return sum / values.length;
    }

    public static void main(String[] args) {
        Integer[] integers = {10, 20, 30, 40};
        Double[] doubles = {1.5, 2.5, 3.5};
        Long[] longs = {100L, 200L, 300L};
```

```
        System.out.println("Integer average: " +
average(integers));
        System.out.println("Double average: " +
average(doubles));
        System.out.println("Long average: " + average(longs));
    }
}
```

Output:



Why does it refuse non-Number types?

A call such as `average(new String[]{"10", "20"})` fails at compile time because `String` does not extend `Number`. This restriction is checked by the compiler, preventing incompatible types from being passed accidentally.

The `Number` bound also lets the method call numeric methods such as `doubleValue()` on each element without casting.

The method rejects empty arrays because their average is undefined. Using `double` is convenient for this exercise, though very large `long` values may lose precision when converted to `double`.

3. (For Advanced Learners – home) Implement a small generic container of your choice (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty case sensibly.

Code:

GenericStack.java

```
class GenericStack<T> {
    private final Object[] elements;
    private int top = -1;

    public GenericStack(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException(
                    "Capacity must be positive");
        }

        elements = new Object[capacity];
    }

    public void push(T value) {
        if (top == elements.length - 1) {
            throw new IllegalStateException("Stack is full");
        }

        elements[++top] = value;
    }

    @SuppressWarnings("unchecked")
    public T pop() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty");
        }
```

```
        T value = (T) elements[top];
        elements[top] = null;
        top--;

        return value;
    }

    @SuppressWarnings("unchecked")
    public T peek() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty");
        }

        return (T) elements[top];
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public int size() {
        return top + 1;
    }
}

public class GenericStackDemo {
    public static void main(String[] args) {
        // Integer stack
        GenericStack<Integer> numbers = new GenericStack<>(5);

        numbers.push(10);
        numbers.push(20);
        numbers.push(30);

        System.out.println("Top number: " + numbers.peek());
```

```
        System.out.println("Removed number: " + numbers.pop());
        System.out.println("Number stack size: " +
numbers.size());

        // String stack
        GenericStack<String> names = new GenericStack<>(5);

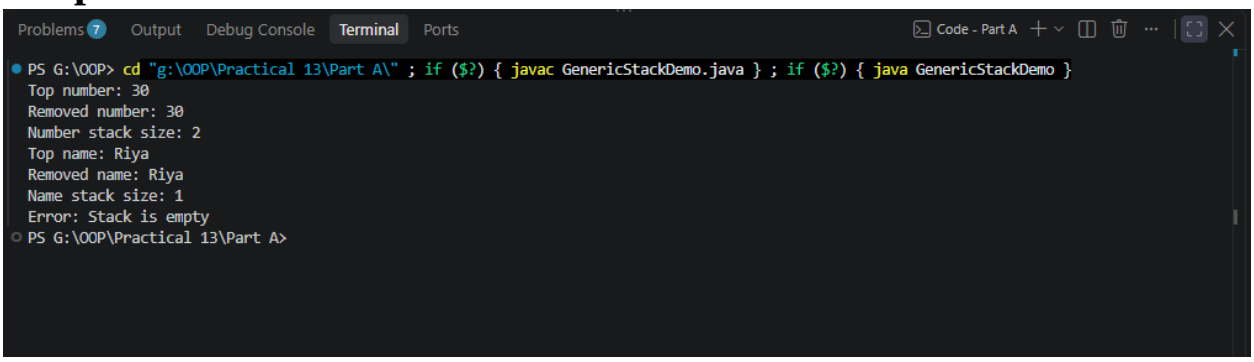
        names.push("Amit");
        names.push("Riya");

        System.out.println("Top name: " + names.peek());
        System.out.println("Removed name: " + names.pop());
        System.out.println("Name stack size: " + names.size());

        // Demonstrate empty-stack handling
        try {
            while (!names.isEmpty()) {
                names.pop();
            }

            names.pop();
        } catch (IllegalStateException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:



Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What problem do generics solve compared with using Object?

Ans.

Generics provide compile-time type safety and reduce the need for explicit type casting. When using Object, a container can store different types, but retrieving a value requires casting, which may cause a ClassCastException at runtime.

2. What is a bounded type parameter, and why is <N extends Number> useful?

Ans.

A bounded type parameter restricts the types that can be used as a generic type argument. The declaration <N extends Number> allows N to be Number or one of its subclasses.

3. What is type erasure?

Ans.

Type erasure is the process by which the Java compiler removes most generic type information during compilation. Unbounded type parameters are generally replaced with Object, while bounded type parameters are replaced with their leftmost bound.